

第二周数据库知识总结：SQLite3 + SQLAlchemy + FastAPI

涵盖：sqlite3 原生操作 → SQLAlchemy 2.0 ORM → FastAPI 集成 CRUD

一、sqlite3 原生操作

1.1 连接数据库

```
python

import sqlite3

# 连接（文件不存在会自动创建）
conn = sqlite3.connect("test.db")

# 内存数据库（程序退出即销毁，适合测试）
conn = sqlite3.connect(":memory:")

# 推荐：用上下文管理器，自动 commit / rollback
with sqlite3.connect("test.db") as conn:
    conn.execute("INSERT INTO users (name) VALUES (?)", ("Alice",))

# 退出 with 块时自动 commit
```

1.2 创建表

```
python

conn = sqlite3.connect("test.db")
cursor = conn.cursor()

cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id    INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT    NOT NULL,
        age   INTEGER
    )
""")

conn.commit()
```

1.3 插入数据（参数化查询，防 SQL 注入）

```
python
```

```
#  正确: 用占位符 ?, 参数以元组传入
cursor.execute("INSERT INTO users (name, age) VALUES (?, ?)", ("Bob", 25))

# 批量插入
users = [("Carol", 30), ("Dave", 22)]
cursor.executemany("INSERT INTO users (name, age) VALUES (?, ?)", users)

conn.commit() # 必须 commit, 否则数据不会写入磁盘

#  错误: f-string 拼接有注入风险
# cursor.execute(f"INSERT INTO users (name) VALUES ({name})")
```

1.4 查询数据

```
python

cursor.execute("SELECT * FROM users WHERE age > ?", (20,))

# 读取一条
row = cursor.fetchone() # 返回 tuple 或 None
print(row)              # (1, 'Bob', 25)

# 读取所有
rows = cursor.fetchall() # 返回 list of tuples

# 迭代 (节省内存)
cursor.execute("SELECT * FROM users")
for row in cursor:
    print(row)

# 让结果可以用列名访问
conn.row_factory = sqlite3.Row
cursor = conn.cursor()
cursor.execute("SELECT * FROM users")
row = cursor.fetchone()
print(row["name"]) # 'Bob'
```

1.5 更新与删除

```
python

cursor.execute("UPDATE users SET age = ? WHERE id = ?", (26, 1))
cursor.execute("DELETE FROM users WHERE id = ?", (1,))
conn.commit()
```

1.6 事务与异常处理

```
python

try:
    conn.execute("INSERT INTO users (name) VALUES (?)", ("Eve",))
    conn.execute("INSERT INTO users (name) VALUES (?)", ("Frank",))
    conn.commit()          # 全部成功才提交
except Exception as e:
    conn.rollback()        # 出错回滚，保持数据一致
    print(f"出错: {e}")
finally:
    conn.close()
```

二、SQLAlchemy 2.0 核心概念

2.1 四个核心对象

对象	职责	类比
Engine	连接池入口，知道如何连数据库	数据库的"总开关"
Connection	一次真实的数据库连接，执行原生 SQL	电话通话本身
Session	ORM 的工作单元，跟踪对象增删改	草稿本，commit 才落地
ORM Model	Python 类 ↔ 数据库表的映射	表的 Python 镜像

2.2 创建 Engine

```
python

from sqlalchemy import create_engine

# SQLite（相对路径）
engine = create_engine("sqlite:///./test.db", echo=True)
# echo=True: 把生成的 SQL 打印到控制台，调试利器，生产环境关掉

# PostgreSQL
# engine = create_engine("postgresql://user:password@localhost/dbname")

# MySQL
# engine = create_engine("mysql+pymysql://user:password@localhost/dbname")
```

2.3 定义 ORM Model（SQLAlchemy 2.0 风格）

```
python

from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from sqlalchemy import String, Integer, ForeignKey
from typing import Optional

# 所有 Model 的基类
class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True, autoincrement=True)
    name: Mapped[str] = mapped_column(String(50), nullable=False)
    age: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)

    def __repr__(self):
        return f"<User id={self.id} name={self.name}>"

class Item(Base):
    __tablename__ = "items"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(100))
    owner_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
```

2.4 创建表

```
python

# 根据所有 Model 定义在数据库中创建表（已存在则跳过）
Base.metadata.create_all(engine)

# 删除所有表（慎用）
# Base.metadata.drop_all(engine)
```

2.5 Session 的三阶段：add → flush → commit

```
python
```

```
from sqlalchemy.orm import Session

with Session(engine) as session:
    user = User(name="Alice", age=28)

    # 第一步: add()
    # 把对象放入 Session 的待处理队列, SQL 还没发出
    session.add(user)
    print(user.id) # None ← 还没插入, 没有 id

    # 第二步: flush()
    # SQL 发给数据库 (INSERT 执行), 但事务未提交
    # 崩溃则回滚; 好处: 可以拿到数据库分配的 id
    session.flush()
    print(user.id) # 1 ← 数据库已分配, 但未持久化

    # 第三步: commit()
    # 事务正式提交, 数据永久写入磁盘
    session.commit()
    print(user.id) # 1 ← 确认持久化
```

三阶段对比：

动作	SQL 发出了吗	事务提交了吗	崩溃会丢吗
add()	否	否	是
flush()	是	否	是
commit()	是	是	否

flush 的实际用途：

```
python

# 插入关联记录时需要先拿到主键
with Session(engine) as session:
    order = Order(product="书")
    session.add(order)
    session.flush() # 让数据库分配 order.id

    item = OrderItem(order_id=order.id, qty=2)
    session.add(item)
    session.commit() # 两条记录一起提交
```

2.6 查询数据

```
python

from sqlalchemy import select

with Session(engine) as session:

    # 按主键查询（2.0 推荐写法）
    user = session.get(User, 1)          # 返回 User 对象或 None

    # 查询所有
    stmt = select(User)
    users = session.execute(stmt).scalars().all()  # list[User]

    # 条件过滤
    stmt = select(User).where(User.age > 20)
    users = session.execute(stmt).scalars().all()

    # 多条件
    stmt = select(User).where(User.age > 20, User.name != "Bob")

    # 排序 + 限制
    stmt = select(User).order_by(User.age.desc()).limit(10)

    # 查询单条（找不到抛异常）
    user = session.execute(select(User).where(User.id == 1)).scalar_one()

    # 查询单条（找不到返回 None）
    user = session.execute(select(User).where(User.id == 1)).scalar_one_or_none()
```

2.7 更新数据

```
python

with Session(engine) as session:
    user = session.get(User, 1)
    if user:
        user.age = 29          # 直接修改属性，Session 会追踪变化
        session.commit()      # 提交时自动生成 UPDATE SQL
```

2.8 删除数据

```
python
```

```
with Session(engine) as session:
    user = session.get(User, 1)
    if user:
        session.delete(user)
        session.commit()
```

2.9 用 Connection 执行原生 SQL

```
python

from sqlalchemy import text

with engine.connect() as conn:
    result = conn.execute(text("SELECT * FROM users WHERE age > :age"), {"age": 20})
    for row in result:
        print(row)
    conn.commit()
```

三、FastAPI + SQLAlchemy 完整 CRUD

3.1 项目结构

```
project/
├── main.py           # FastAPI 路由
├── database.py       # Engine + Session 配置
├── models.py        # ORM Model
└── schemas.py       # Pydantic 数据验证模型
```

3.2 database.py — 数据库配置

```
python
```

```

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, Session

DATABASE_URL = "sqlite:///./app.db"

engine = create_engine(
    DATABASE_URL,
    connect_args={"check_same_thread": False}, # SQLite 专用, 允许多线程
    echo=True
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# 依赖函数: 每个请求得到独立 Session, 请求结束自动关闭
def get_db():
    db = SessionLocal()
    try:
        yield db          # 把 Session 交给路由函数
    finally:
        db.close()       # 无论成功还是报错都会执行

```

3.3 models.py — ORM Model

```

python

from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from sqlalchemy import String

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(primary_key=True, index=True)
    name: Mapped[str] = mapped_column(String(50))
    email: Mapped[str] = mapped_column(String(100), unique=True)

```

3.4 schemas.py — Pydantic 数据验证

```

python

```



```
from pydantic import BaseModel

# 创建时的请求体
class UserCreate(BaseModel):
    name: str
    email: str

# 返回给客户端的数据结构
class UserOut(BaseModel):
    id: int
    name: str
    email: str

model_config = {"from_attributes": True} # 允许从 ORM 对象转换
```

3.5 main.py — 完整 CRUD 路由

python

```

from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session
from sqlalchemy import select

from database import engine, get_db
from models import Base, User
from schemas import UserCreate, UserOut

Base.metadata.create_all(engine) # 启动时建表

app = FastAPI()

# — CREATE —————
@app.post("/users/", response_model=UserOut)
def create_user(user: UserCreate, db: Session = Depends(get_db)):
    db_user = User(name=user.name, email=user.email)
    db.add(db_user)
    db.commit()
    db.refresh(db_user) # 从数据库同步最新字段（如自增 id）
    return db_user

# — READ LIST —————
@app.get("/users/", response_model=list[UserOut])
def read_users(skip: int = 0, limit: int = 10, db: Session = Depends(get_db)):
    stmt = select(User).offset(skip).limit(limit)
    return db.execute(stmt).scalars().all()

# — READ ONE —————
@app.get("/users/{user_id}", response_model=UserOut)
def read_user(user_id: int, db: Session = Depends(get_db)):
    user = db.get(User, user_id)
    if not user:
        raise HTTPException(status_code=404, detail="用户不存在")
    return user

# — UPDATE —————
@app.put("/users/{user_id}", response_model=UserOut)
def update_user(user_id: int, data: UserCreate, db: Session = Depends(get_db)):
    user = db.get(User, user_id)
    if not user:
        raise HTTPException(status_code=404, detail="用户不存在")
    user.name = data.name
    user.email = data.email
    db.commit()
    db.refresh(user)
    return user

```

```
# — DELETE —————
@app.delete("/users/{user_id}")
def delete_user(user_id: int, db: Session = Depends(get_db)):
    user = db.get(User, user_id)
    if not user:
        raise HTTPException(status_code=404, detail="用户不存在")
    db.delete(user)
    db.commit()
    return {"message": f"用户 {user_id} 已删除"}
```

3.6 Depends(get_db) 的工作原理

```
python

# get_db 是一个 generator 函数 (含 yield)
def get_db():
    db = SessionLocal()    # 1. 请求进来: 创建 Session
    try:
        yield db           # 2. 把 Session 注入路由函数, 等待请求处理完
    finally:
        db.close()         # 3. 请求结束 (成功或报错): 关闭 Session

# Depends 保证:
# [✓] 每个请求有独立的 Session (不共享)
# [✓] 请求结束后 Session 自动关闭, 不泄漏连接
# [✓] 路由函数抛异常时, finally 仍会执行
```

四、易混淆对比

4.1 ORM Model vs Pydantic Schema

	ORM Model	Pydantic Schema
继承自	<code>Base</code> (<code>DeclarativeBase</code>)	<code>BaseModel</code>
用途	映射数据库表, 执行增删改查	验证请求/响应数据, 序列化
与数据库的关系	直接对应表结构	与数据库无关
典型文件	<code>models.py</code>	<code>schemas.py</code>

4.2 session.get() vs select()

```
python

# session.get: 按主键查，有缓存，适合已知 id
user = session.get(User, 1)

# select: 灵活查询，支持任意条件
stmt = select(User).where(User.name == "Alice")
user = session.execute(stmt).scalar_one_or_none()
```

4.3 commit() vs refresh()

```
python

db.commit()      # 提交事务，数据写入数据库
db.refresh(user) # commit 之后用：把数据库最新值同步回 Python 对象
                # 典型场景：获取数据库自动生成的 id、默认时间戳
```

五、常见错误速查

```
python
```

```
# ❌ 忘记 commit
db.add(user)
return user          # 数据没有真正写入!

# ✅ 正确
db.add(user)
db.commit()
db.refresh(user)
return user

# ❌ SQLite 多线程不加参数
engine = create_engine("sqlite:///./app.db")

# ✅ SQLite 需要加 check_same_thread=False
engine = create_engine("sqlite:///./app.db", connect_args={"check_same_thread": False})

# ❌ Pydantic v2 忘记配置 from_attributes
class UserOut(BaseModel):
    id: int
    name: str
# 从 ORM 对象转换时会报错

# ✅ 需要加 model_config
class UserOut(BaseModel):
    id: int
    name: str
    model_config = {"from_attributes": True}
```

文档覆盖 : *sqlite3 · SQLAlchemy 2.0 · FastAPI CRUD · Session 生命周期 · Pydantic 集成*